MINI PROJECT REPORT

STUDENT RECORD SEARCH SYSTEM USING TREES

Submitted By: Lakshya

Roll Number: 2556470

Course: BTech CSE A

Subject: Data Structures and Algorithms

1. INTRODUCTION

The Student Record Search System is a mini project developed using the concepts of Trees in Data Structures and Algorithms. It stores student records in a Binary Search Tree (BST) in which the enrolment ID of the student is the key.

The system supports insertion of records, rejection of duplicate enrolment IDs, searching by enrolment ID, deletion of records, and display of records using inorder, preorder and postorder traversals. The inorder traversal displays the records in sorted order of enrolment ID. The project also compares the number of comparisons and the tree height for a balanced-looking insertion order and a sorted insertion order, to show how the shape of a BST affects its efficiency.

The project is implemented in the C programming language using a menu-driven interface.

2. OBJECTIVES

The main objectives of this project are:

- To understand the Binary Search Tree and its ordering property.
- To store student records using an enrolment ID as a unique key.
- To implement insertion (rejecting duplicate IDs), searching and deletion in a BST.
- To implement inorder, preorder and postorder traversals.
- To handle the three deletion cases: leaf node, node with one child and node with two children.
- To compare balanced and skewed trees using the number of comparisons and the height of the tree.
- To explain the average-case and worst-case time complexity of BST operations and the role of self-balancing trees such as the AVL tree.

3. PROBLEM STATEMENT

A student record system must be able to add new students, find a student quickly, remove a student and list all students in a useful order. Searching in an unsorted list takes time proportional to the number of students. This project uses a Binary Search Tree so that records can be inserted, searched and deleted efficiently, and displayed in sorted order of enrolment ID. Each record contains a unique enrolment ID, the student name and the department. Duplicate enrolment IDs are rejected with a clear message.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

- Programming Language: C
- Compiler: GCC
- Editor: Visual Studio Code with the integrated terminal
- Operating System: macOS (the program also runs on Windows and Linux)

Hardware Requirements

- Computer or laptop
- Basic memory and storage
- Keyboard and display

5. DATA STRUCTURE USED

A Binary Search Tree is a binary tree in which every node has at most two children, and the keys are arranged according to the BST ordering property:

- All keys in the left subtree of a node are smaller than the key of the node.
- All keys in the right subtree of a node are greater than the key of the node.
- No two nodes have the same key (duplicate enrolment IDs are not allowed).

Each node of the tree is a student record. It is stored in a C structure with the following fields:

Field	Type	Purpose
id	int	Enrolment ID (used as the BST key)
name	char array (50)	Name of the student
dept	char array (50)	Programme or department of the student
left	pointer to node	Pointer to the left child (smaller IDs)
right	pointer to node	Pointer to the right child (greater IDs)

The tree is wrapped in a small structure that stores the root pointer and a counter of the key comparisons made during insertion. Nodes are created dynamically using malloc, and all nodes are released using free before the program exits.

Because the left subtree always holds smaller keys and the right subtree larger keys, an inorder traversal (left, node, right) visits the records in increasing order of enrolment ID.

6. ALGORITHMS USED

6.1 Insertion

1. If the current position is empty (NULL), create a new node here.
2. Otherwise compare the new ID with the ID of the current node (one comparison is counted).
3. If the new ID is smaller, move to the left child; if it is greater, move to the right child.
4. If the IDs are equal, the insertion is rejected and the message "Duplicate! Enrolment ID already exists" is displayed.

6.2 Search

The search follows the same path as insertion. Starting from the root, the ID is compared with the current node; the search goes left if the ID is smaller and right if it is greater. If a node with the same ID is found, the ID, name and department are displayed. If a NULL position is reached, the student is not found.

6.3 Deletion

The node with the given ID is first located using the search path. Then one of three cases is applied:

1. **Leaf node (no children):** the node is simply removed and its parent pointer becomes NULL.
2. **Node with one child:** the node is removed and its only child takes its place.
3. **Node with two children:** the inorder successor (the smallest node in the right subtree) is found, its data is copied into the node to be deleted, and then the successor node is deleted from the right subtree.

The program prints which case was applied, for example "(Case: Leaf node)".

6.4 Traversals

- **Inorder (left, node, right):** displays the records in sorted order of enrolment ID.
- **Preorder (node, left, right):** visits the node before its subtrees; useful for copying the tree structure.
- **Postorder (left, right, node):** visits the node after its subtrees; useful for deleting a tree.

6.5 Height of the Tree and Comparison of Insertion Orders

The height of the tree is calculated recursively as 1 plus the larger of the heights of the left and right subtrees. To compare tree shapes, the program inserts the same 15 keys into two separate trees, one in a balanced-looking order (50, 30, 70, 20, 40, 60, 80, ...) and one in sorted order (5, 10, 20, 25, ...), and displays the total number of comparisons and the final height of each tree.

7. SYSTEM IMPLEMENTATION

The program is implemented in C and provides the following menu options:

Option	Function
1	Insert student
2	Search student by enrolment ID
3	Delete student
4	Inorder traversal
5	Preorder traversal
6	Postorder traversal
7	Load 10 sample records
8	Compare balanced and sorted insertion
9	Show tree height
10	Count students
11	Display insertion comparisons
12	Reset comparison counter
0	Exit

For insertion, the user enters the enrolment ID, the name and the department. For searching and deletion, the user enters the enrolment ID. Option 7 inserts ten sample records (IDs 50, 30, 70, 20, 40, 60, 80, 35, 45 and 65) so that the tree has at least ten records for testing. Before exiting, the program frees all dynamically allocated nodes.

8. TEST CASES

The following test cases were executed in the run shown in the output screenshots. In this run, five records were entered by the user in increasing order of enrolment ID (2556470, 2556471, 2556472, 2556512 and 25564517), one more record (2556457) was inserted and then deleted, and the ten sample records were loaded afterwards.

Test	Description	Input	Expected Output	Actual Output	Result
1	Insertion of records	Option 1 with IDs 2556470, 2556471, 2556472, 2556512, 25564517 and 2556457	Each record is inserted and an "Inserted" message is displayed	"Inserted: ID - Name" displayed for every record	PASS
2	Successful search	Option 2, ID 25564517	Details of the student with this ID	Student Found! ID: 25564517, Name: Vinit, Department: CSE	PASS
3	Deletion of a leaf node	Option 3, ID 2556457 (leaf node)	Leaf case is applied and the student is deleted	(Case: Leaf node) Student deleted successfully.	PASS

Test	Description	Input	Expected Output	Actual Output	Result
4	Inorder traversal	Option 4	Records in increasing order of ID	2556470, 2556471, 2556472, 2556512, 25564517 (sorted)	PASS
5	Preorder and postorder traversal	Options 5 and 6	For this right-skewed tree, preorder is the same as inorder and postorder is the reverse	Preorder: 2556470 to 25564517 in increasing order. Postorder: 25564517 to 2556470 in decreasing order	PASS
6	Balanced vs sorted insertion	Option 8 (15 keys)	Sorted order needs more comparisons and gives a greater height	Balanced: 35 comparisons, height 5. Sorted: 105 comparisons, height 15	PASS
7	Count, height and comparison counter	Options 10, 9, 11 and 12	5 + 10 = 15 students after loading the samples; counter can be displayed and reset	Total students: 15. Tree height: 5. Insertion comparisons: 40. Counter reset message displayed	PASS

Summary: All 7 executed test cases passed.

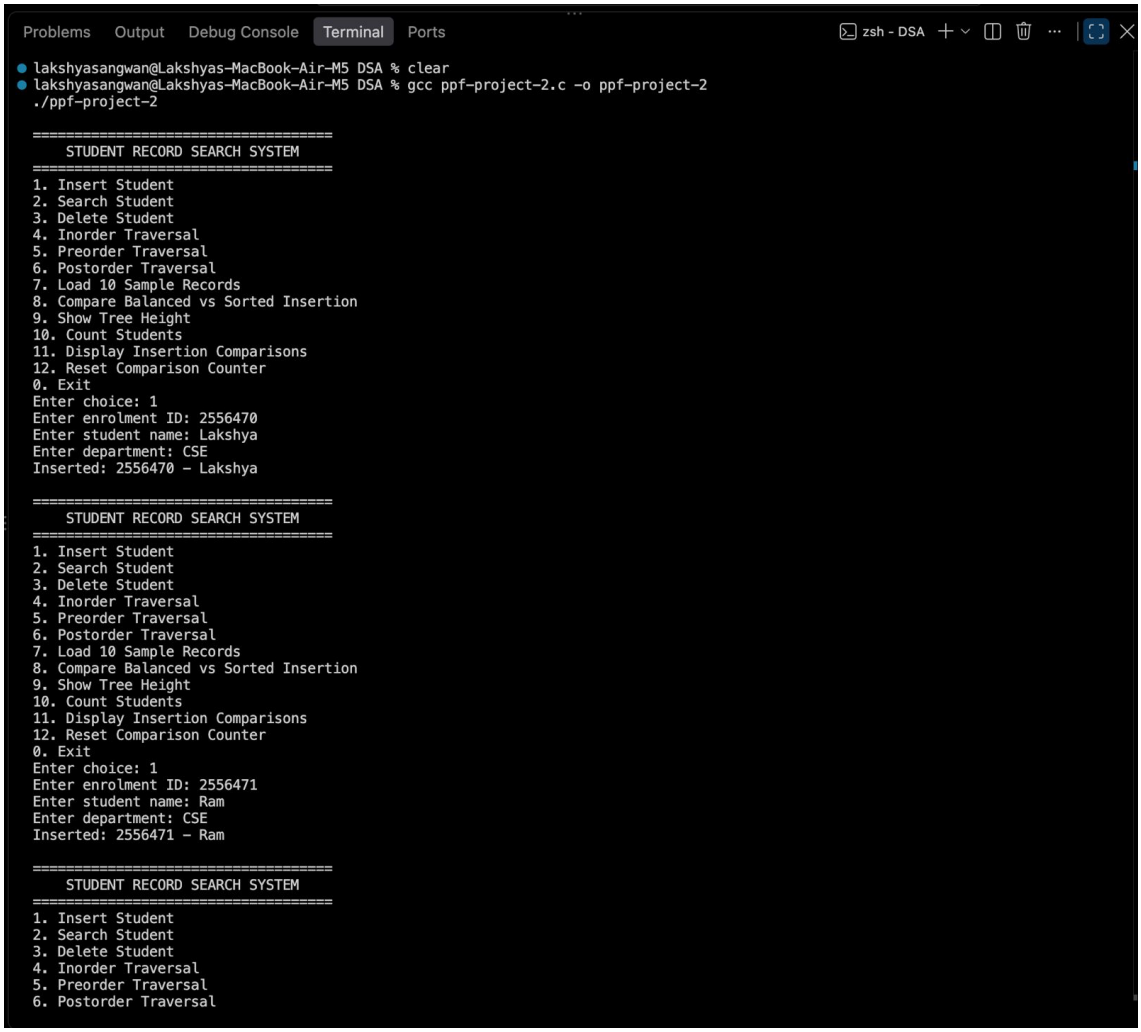
Explanation of the outputs: The five records entered by the user were inserted in increasing order of enrolment ID, so each new record became the right child of the previous one. The tree at that point was therefore a right-skewed chain, and this is why the preorder output is the same as the inorder output and the postorder output is the reverse. The tree height of 5 shown after loading the samples comes from this chain of five records.

The value 40 shown for insertion comparisons is the total number of key comparisons made while inserting all 16 records of this run in the main tree. The comparisons made by option 8 are counted separately in its own two temporary trees.

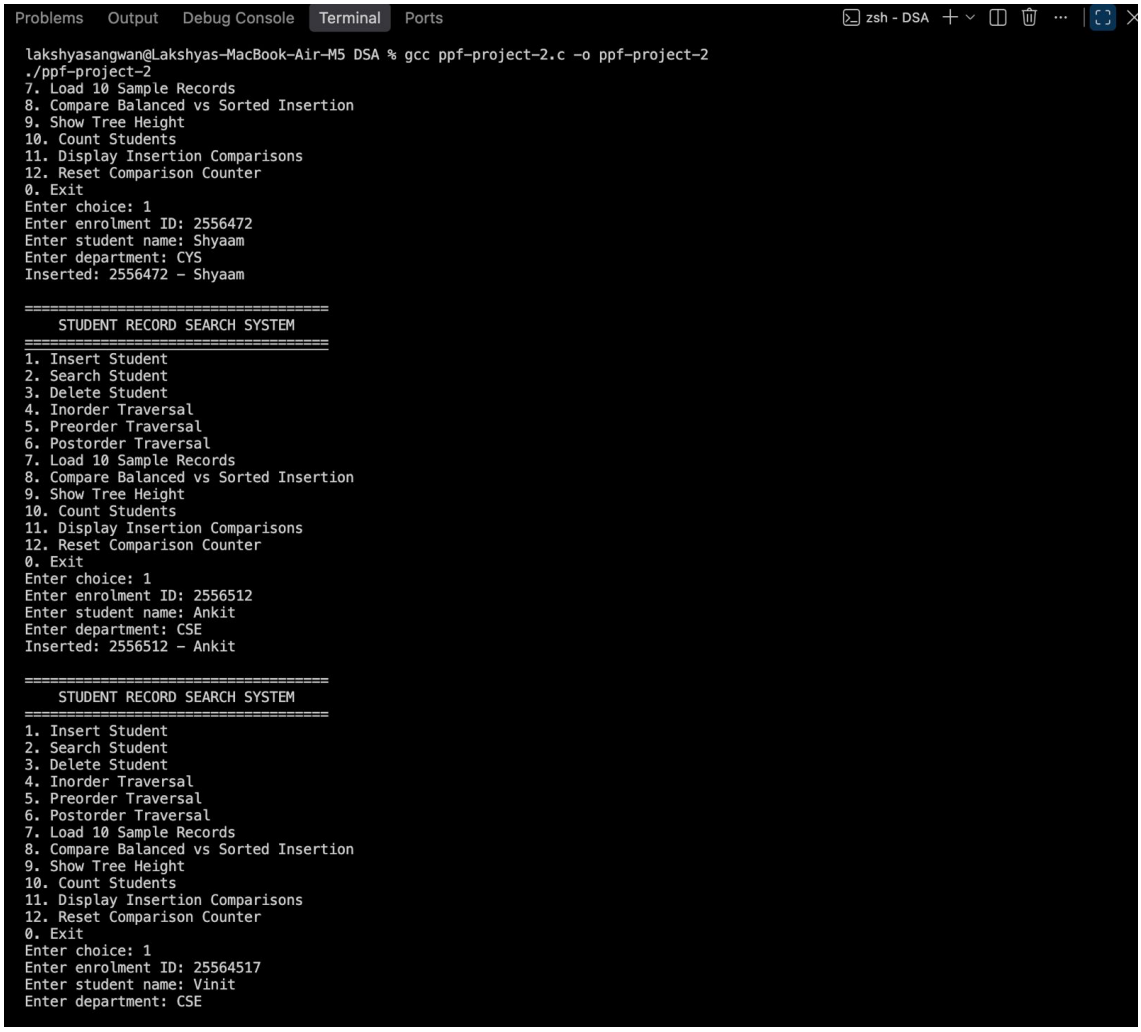
The deletion of a leaf node is shown in Screenshot 4. The deletion cases for a node with one child and a node with two children are explained in Section 6.3.

9. OUTPUT SCREENSHOTS

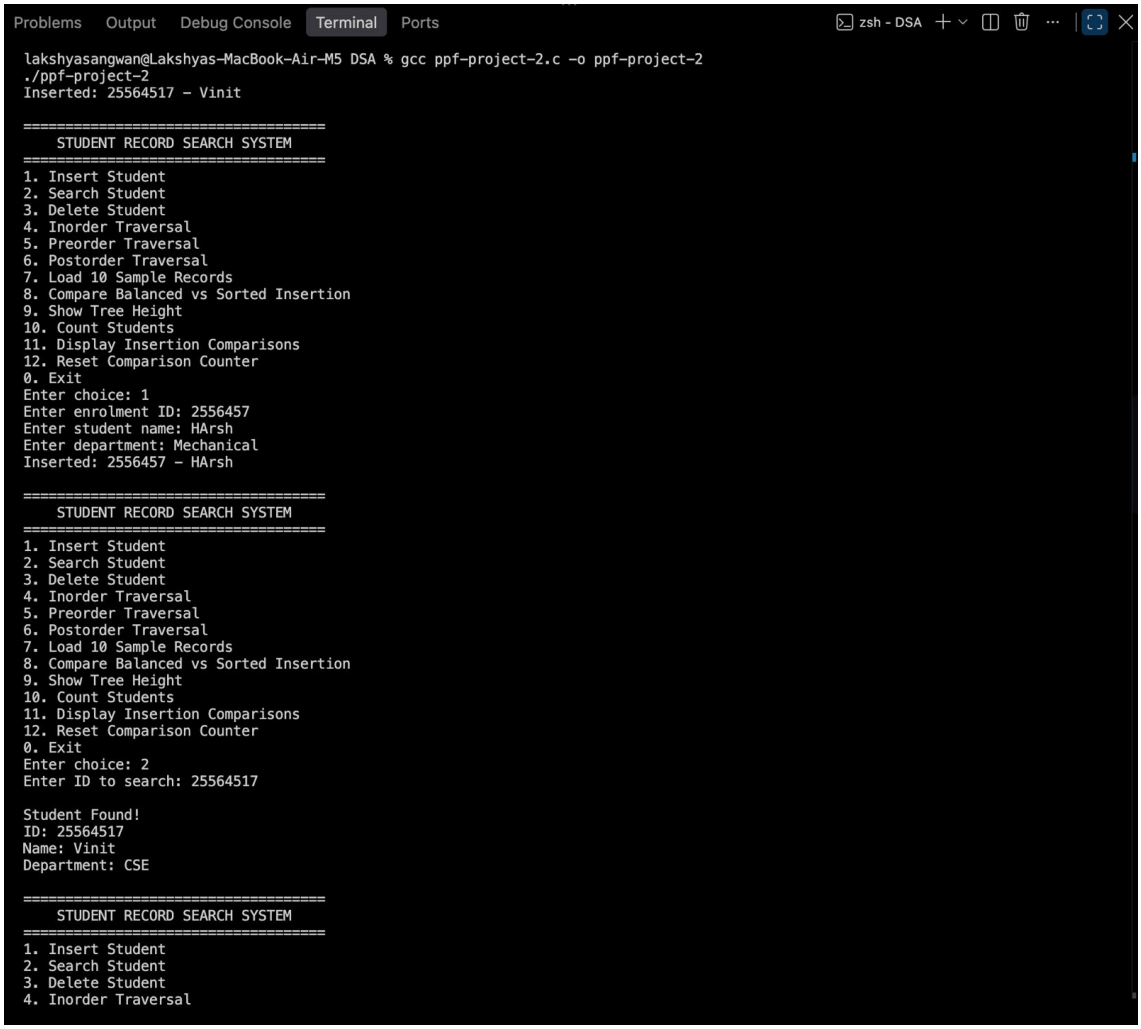
Screenshot 1: Main menu and insertion of records 2556470 (Lakshya) and 2556471 (Ram) using option 1



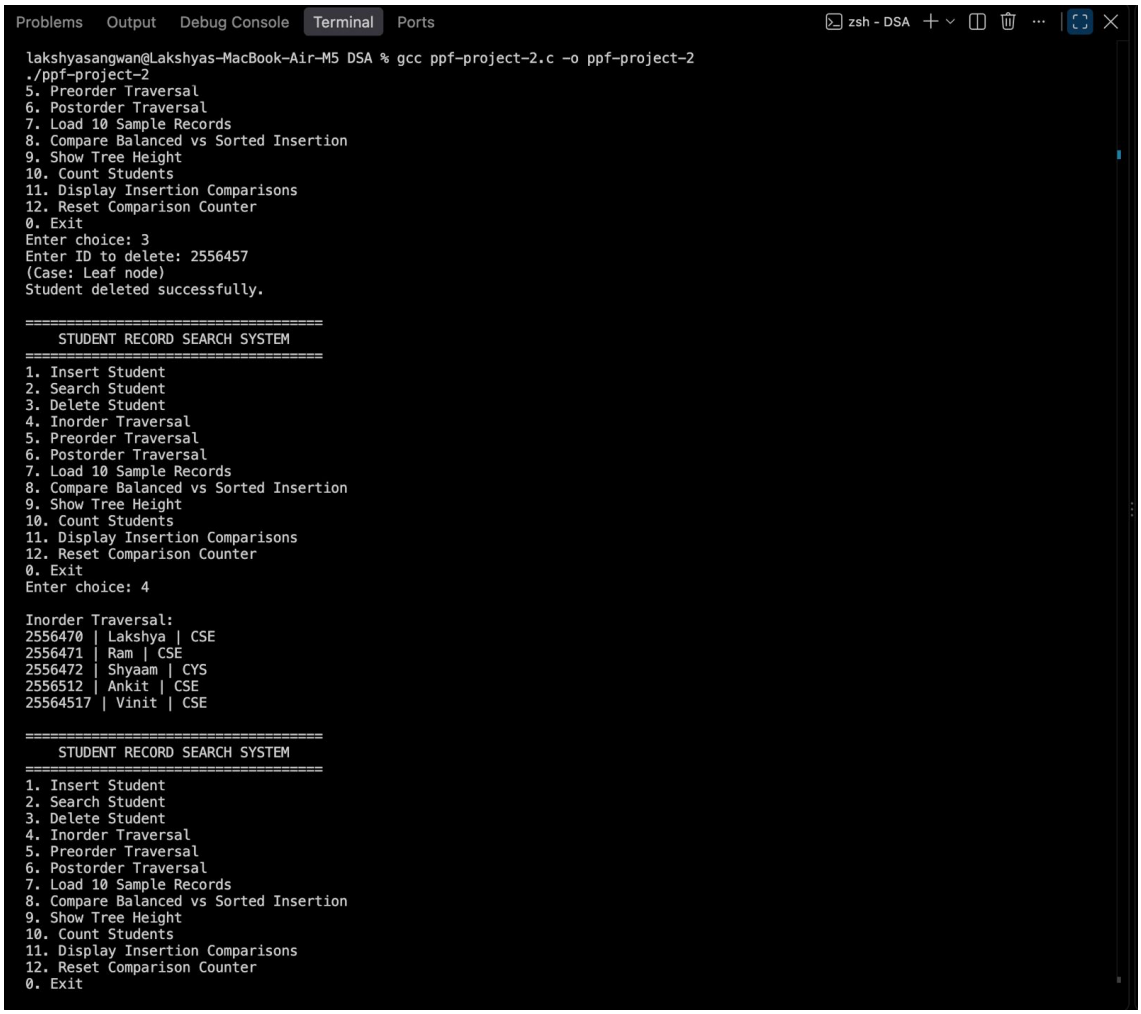
Screenshot 2: Insertion of records 2556472 (Shyaam) and 2556512 (Ankit); details of the next record being entered



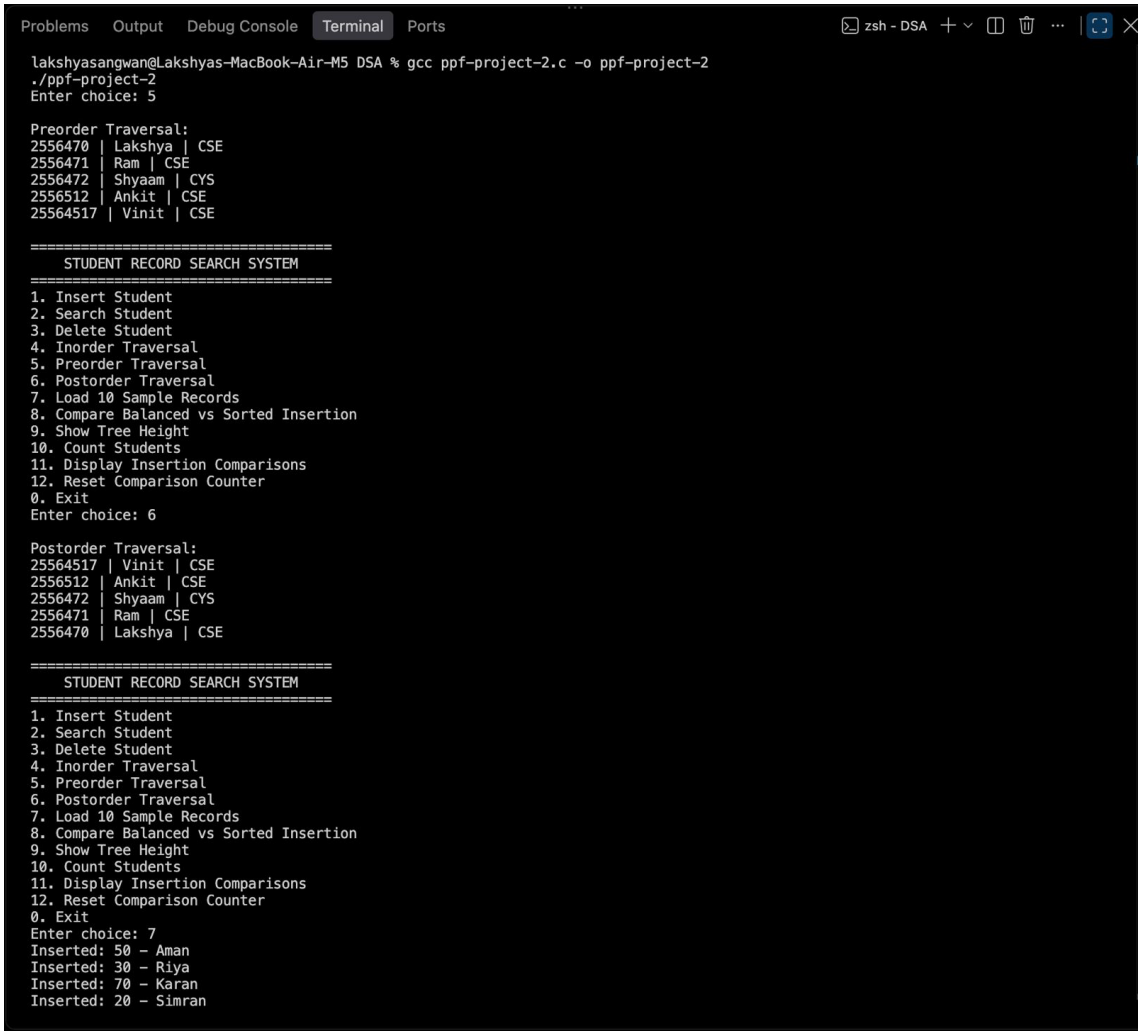
Screenshot 3: Insertion of 25564517 (Vinit) and 2556457 (Harsh), and successful search by enrolment ID (option 2)



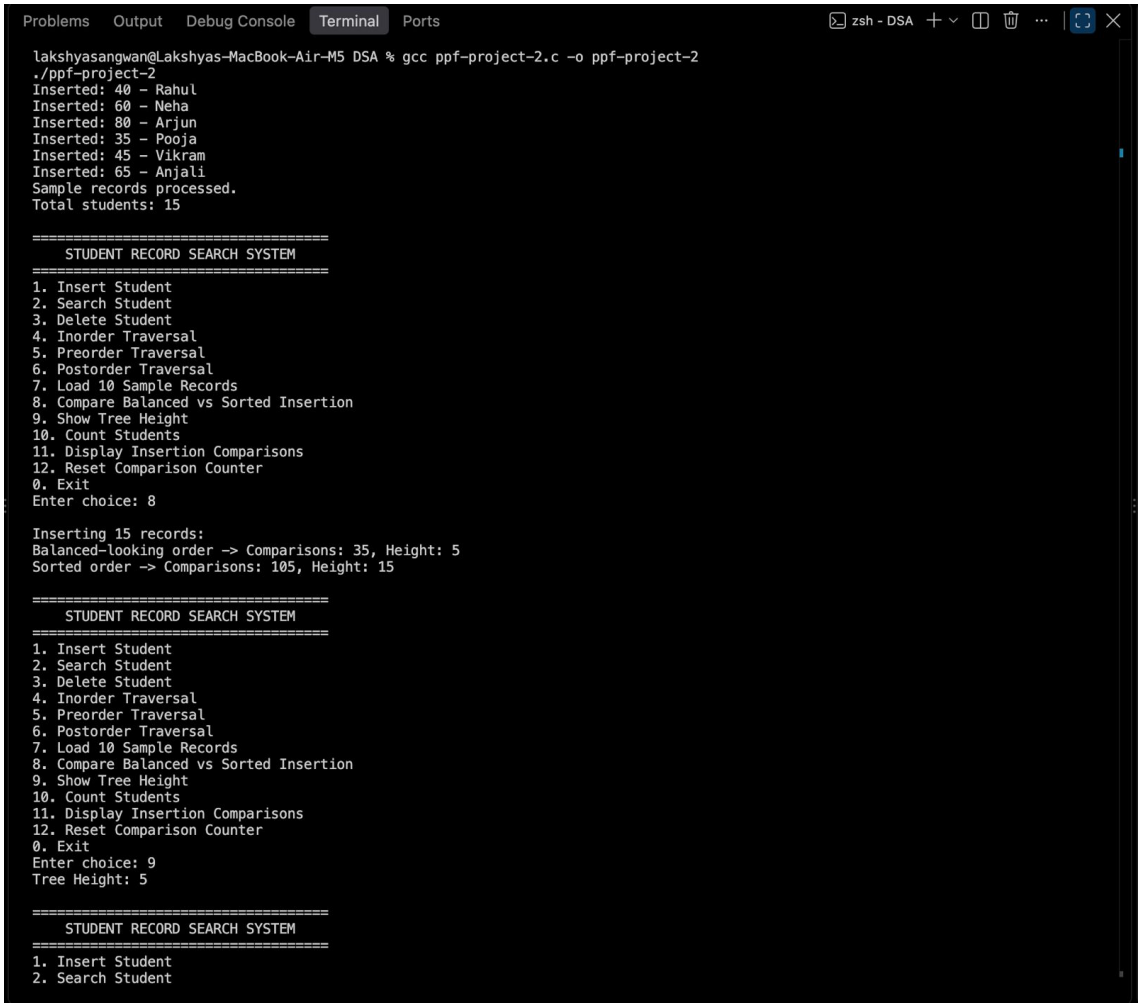
Screenshot 4: Deletion of a leaf node (option 3) and inorder traversal (option 4)



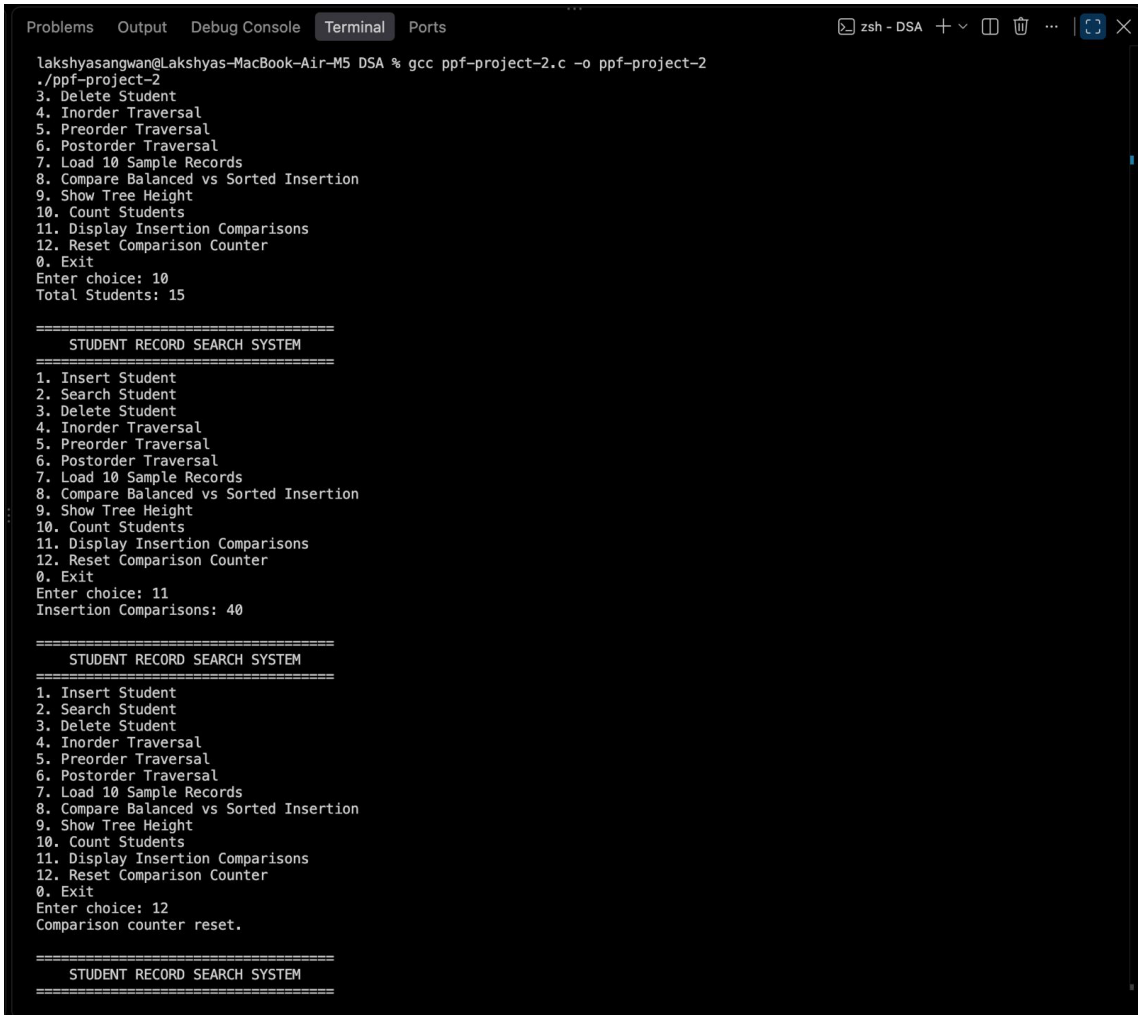
Screenshot 5: Preorder traversal (option 5), postorder traversal (option 6) and loading of sample records (option 7)



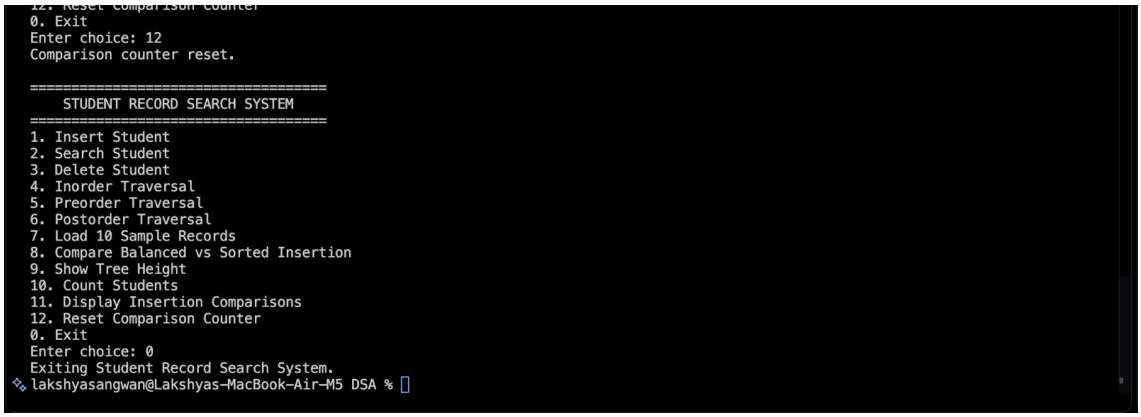
Screenshot 6: Remaining sample records, total students, comparison of balanced and sorted insertion (option 8) and tree height (option 9)



Screenshot 7: Count of students (option 10), insertion comparisons (option 11) and reset of the comparison counter (option 12)



Screenshot 8: Exit from the program (option 0)



10. TIME AND SPACE COMPLEXITY

Let n be the number of records in the tree and h the height of the tree.

Operation	Average Case	Worst Case
Insertion	O(log n)	O(n)
Search	O(log n)	O(n)
Deletion	O(log n)	O(n)
Inorder, preorder, postorder traversal	O(n)	O(n)
Height calculation	O(n)	O(n)

Each insertion, search or deletion follows one path from the root towards a leaf, so it takes time proportional to the height h of the tree. Traversals visit every node once, so they take O(n) time.

Space complexity: O(n) to store n records. The recursion used by the operations needs O(h) additional stack space.

10.1 Why a Regular BST Can Degrade to O(n)

A regular BST does not control its own shape. If the keys are inserted in already sorted order, every new key is greater than all existing keys and goes to the right of the last node. The tree becomes a skewed tree, which is like a linked list, and its height becomes n. Every operation must then follow a path of n nodes, so the time becomes O(n). The program demonstrates this with option 8: for 15 keys, the balanced-looking order gives a height of 5 and needs 35 comparisons, whereas the sorted order gives a height of 15 and needs 105 comparisons.

10.2 How a Self-Balancing Tree (AVL Tree) Solves This

An AVL tree is a self-balancing BST. For every node, the balance factor (height of the left subtree minus the height of the right subtree) is kept as -1, 0 or 1. After each insertion or deletion, if a node becomes unbalanced, the tree is repaired using rotations (LL, RR, LR and RL rotations). This keeps the height of the tree at O(log n) even when keys are inserted in sorted order, so insertion, search and deletion always take O(log n) time.

11. ADVANTAGES

- Simple and easy-to-use menu-driven interface.
- Records can be searched, inserted and deleted efficiently when the tree is reasonably balanced.
- Records are displayed in sorted order of enrolment ID using inorder traversal.
- Duplicate enrolment IDs are rejected with a clear message.
- All three deletion cases are handled, and the case used is displayed.
- Demonstrates the effect of tree shape using the comparison count and height.
- Dynamically allocated memory is released before the program exits.

12. LIMITATIONS

- A regular BST is not self-balancing, so sorted input makes it skewed and the operations take O(n) time.
- Records are stored only in memory and are lost when the program exits.

- Records cannot be edited; a record must be deleted and inserted again to change it.
- Search is possible only by enrolment ID, not by name or department.
- Names and departments are limited to 49 characters.

13. FUTURE SCOPE

The project can be improved by:

- Using an AVL tree or a Red-Black tree to keep the tree balanced.
- Saving and loading the records from a file.
- Adding an option to update an existing record.
- Adding search by name or department.
- Developing a graphical interface.

14. CONCLUSION

The Student Record Search System shows how a Binary Search Tree can be used to store and manage student records using the enrolment ID as a key. The project implements insertion with duplicate rejection, searching, deletion with all three cases, and the three traversals, and uses inorder traversal to display the records in sorted order. The comparison of balanced and sorted insertion (35 comparisons and height 5 against 105 comparisons and height 15 for 15 keys) clearly shows that the shape of the tree decides the efficiency of a BST, and that self-balancing trees such as the AVL tree are needed to guarantee O(log n) operations.

This project provided practical experience with trees, recursion, pointers, dynamic memory allocation and menu-driven programming in C.

15. REFERENCES

1. Data Structures and Algorithms course notes.
2. C Programming Language reference materials.
3. Binary Search Tree, tree traversal and AVL tree concepts.